ΣΥΣΤΗΜΑΤΑ ΔΙΑΧΕΙΡΙΣΗΣ ΜΕΓΑΛΟΥ ΟΓΚΟΥ ΔΕΔΟΜΕΝΩΝ

ΤΖΟΥΝΙΔΗΣ ΔΗΜΗΤΡΙΟΣ – ΑΜ: 2121241

(όλοι οι σχετικοί κώδικες υπάρχουν στον παρακάτω σύνδεσμο GitHub: https://github.com/Dimjoun/ny-taxi-spark-analytics)

1. Εισαγωγή

Η παρούσα εργασία εκπονήθηκε στο πλαίσιο του μαθήματος «Συστήματα Διαχείρισης Μεγάλου Όγκου Δεδομένων» και έχει ως στόχο την επεξεργασία και ανάλυση δεδομένων μεγάλου όγκου με χρήση του Apache Spark σε περιβάλλον κατανεμημένων συστημάτων.

Αντικείμενο της εργασίας είναι η υλοποίηση και αξιολόγηση διαφορετικών queries πάνω σε δεδομένα ταξί της Νέας Υόρκης (NY Yellow Taxi), για τα έτη 2015 και 2024. Τα δεδομένα αποθηκεύτηκαν στο σύστημα αρχείων HDFS και επεξεργάστηκαν μέσω Spark APIs, χρησιμοποιώντας διαφορετικές προσεγγίσεις όπως DataFrames και Spark SQL.

Στο πλαίσιο της εργασίας πραγματοποιήθηκε σύγκριση διαφορετικών μορφών αποθήκευσης δεδομένων (CSV και Parquet), καθώς και ανάλυση των στρατηγικών εκτέλεσης που επιλέγει ο Spark Catalyst Optimizer για την εκτέλεση joins και aggregations. Επιπλέον, καταγράφηκαν οι χρόνοι εκτέλεσης των queries και παρουσιάστηκαν τα

αποτελέσματα σε μορφή πινάκων και γραφημάτων, ώστε να εξαχθούν συμπεράσματα σχετικά με την απόδοση κάθε προσέγγισης.

2. Περιγραφή Υλοποίησης

Η υλοποίηση της εργασίας πραγματοποιήθηκε με χρήση του Apache Spark σε περιβάλλον Kubernetes, αξιοποιώντας το κατανεμημένο σύστημα αρχείων Hadoop (HDFS) για την αποθήκευση των δεδομένων.

Τα δεδομένα που χρησιμοποιήθηκαν προέρχονται από το dataset **NY Yellow Taxi**, το οποίο περιλαμβάνει πληροφορίες για διαδρομές ταξί, όπως:

- ημερομηνία και ώρα διαδρομής
- απόσταση διαδρομής
- σημεία επιβίβασης και αποβίβασης
- γεωγραφικές περιοχές (zones και boroughs)

Αρχικά, τα δεδομένα αποθηκεύτηκαν σε μορφή CSV και στη συνέχεια μετατράπηκαν σε μορφή Parquet, προκειμένου να αξιολογηθεί η επίδραση της μορφής αποθήκευσης στην απόδοση των queries.

Στην εργασία υλοποιήθηκαν έξι διαφορετικά queries (Q1–Q6), τα οποία εκτελέστηκαν με διαφορετικές τεχνικές επεξεργασίας δεδομένων. Για κάθε query καταγράφηκαν οι χρόνοι εκτέλεσης μέσω του Spark History Server και δημιουργήθηκαν γραφήματα σύγκρισης των αποτελεσμάτων.

3. Q1 – Φιλτράρισμα και Ομαδοποίηση

Το Query 1 έχει ως στόχο την εκτέλεση φιλτραρίσματος και ομαδοποίησης των δεδομένων διαδρομών ταξί, με σκοπό τον υπολογισμό του πλήθους των διαδρομών που ικανοποιούν συγκεκριμένα κριτήρια. Η επεξεργασία πραγματοποιήθηκε με χρήση του Spark DataFrame API και εφαρμόστηκαν λειτουργίες filtering και aggregation.

Για την αξιολόγηση της απόδοσης του query καταγράφηκαν οι χρόνοι εκτέλεσης μέσω του Spark History Server.

Χρόνος Εκτέλεσης

Υλοποίηση	**Χρόνος**
Q1_DF_CSV	7.8 min
Q1_DF_CSV (run 2)	9 s
Q1_RDD_Parquet	4.3 min
Q1_DF_Parquet_UDF	50 s
Q1_DF_Parquet	54 s

Ανάλυση

Η εκτέλεση του Query 1 έδειξε ότι η χρήση της μορφής αποθήκευσης Parquet οδηγεί σε σημαντική βελτίωση της απόδοσης σε σχέση με τα CSV αρχεία. Αυτό οφείλεται στο γεγονός ότι τα Parquet αρχεία είναι columnar και επιτρέπουν στο Spark να διαβάζει μόνο τις απαραίτητες στήλες.

Επιπλέον, η χρήση UDF μπορεί να επηρεάσει την απόδοση, καθώς παρακάμπτει ορισμένες από τις βελτιστοποιήσεις του Spark Catalyst Optimizer.

Συμπέρασμα

Η χρήση του DataFrame API σε συνδυασμό με αρχεία Parquet αποτελεί την πιο αποδοτική προσέγγιση για την εκτέλεση του Query 1, καθώς μειώνει σημαντικά τον χρόνο εκτέλεσης και τη χρήση πόρων του συστήματος.

4. Q2 – Υπολογισμός Μοναδικών Τιμών / Μέγιστης Απόστασης

Το Query 2 έχει ως στόχο τον υπολογισμό συγκεκριμένων στατιστικών για τις διαδρομές ταξί, αξιοποιώντας δεδομένα από τα έτη 2015 και 2024. Η επεξεργασία των δεδομένων πραγματοποιήθηκε με χρήση του Apache Spark και εφαρμόστηκαν λειτουργίες filtering, υπολογισμού αποστάσεων και ομαδοποίησης.

Για την εκτέλεση του query χρησιμοποιήθηκαν διαφορετικές προσεγγίσεις επεξεργασίας δεδομένων, με στόχο τη σύγκριση της απόδοσης μεταξύ διαφορετικών υλοποιήσεων και μορφών αποθήκευσης δεδομένων.

Οι χρόνοι εκτέλεσης καταγράφηκαν μέσω του Spark History Server.

Χρόνος Εκτέλεσης

Υλοποίηση	Χρόνος
Q2_RDD_Parquet	9.7 min
Q2_SQL_Parquet	1.7 min
Q2_DF_Parquet	3.1 min
Q2_SQL_Parquet (run 2)	1.4 min

Ανάλυση

Η εκτέλεση του Query 2 έδειξε ότι η χρήση της SQL πάνω σε δεδομένα Parquet παρέχει σημαντικά καλύτερη απόδοση σε σχέση με άλλες υλοποιήσεις. Αυτό οφείλεται στο γεγονός ότι ο Spark Catalyst Optimizer μπορεί να εφαρμόσει βελτιστοποιήσεις, όπως:

- projection pushdown
- optimized aggregation
- efficient execution planning

Αντίθετα, η υλοποίηση με RDD παρουσίασε μεγαλύτερο χρόνο εκτέλεσης, καθώς δεν αξιοποιεί πλήρως τις βελτιστοποιήσεις του Spark.

Η χρήση του Parquet format συνέβαλε επίσης στη βελτίωση της απόδοσης, καθώς επιτρέπει την αποδοτικότερη ανάγνωση των δεδομένων.

Συμπέρασμα

Η χρήση της SQL σε συνδυασμό με αρχεία Parquet αποτελεί την πιο αποδοτική προσέγγιση για την εκτέλεση του Query 2. Η επιλογή κατάλληλου API και μορφής αποθήκευσης δεδομένων επηρεάζει σημαντικά τον χρόνο εκτέλεσης και την αποδοτικότητα της επεξεργασίας.

5. Q3 – Join και Aggregation

Το Query 3 έχει ως στόχο τον υπολογισμό του αριθμού διαδρομών που ξεκινούν και καταλήγουν στην ίδια γεωγραφική περιοχή (Borough). Για την εκτέλεση του query χρησιμοποιήθηκαν δεδομένα από το αρχείο διαδρομών ταξί καθώς και ο πίνακας αντιστοίχισης γεωγραφικών περιοχών (taxi_zone_lookup).

Η υλοποίηση του query πραγματοποιήθηκε με διαφορετικές προσεγγίσεις επεξεργασίας δεδομένων, χρησιμοποιώντας τόσο τη μορφή CSV όσο και τη μορφή Parquet. Στόχος ήταν η σύγκριση της απόδοσης των διαφορετικών formats και η αξιολόγηση των στρατηγικών execution που επιλέγει ο Spark Catalyst Optimizer.

Οι χρόνοι εκτέλεσης καταγράφηκαν μέσω του Spark History Server.

Χρόνος Εκτέλεσης

Υλοποίηση	Χρόνος
Q3_DF_CSV	7.2 min
Q3_SQL_Parquet	1.8 min
Q3_DF_Parquet	1.9 min
Q3_DF_Parquet_Pruning	30 s
Q3_DF_Parquet_Pruning (run 2)	25 s

Ανάλυση

Η εκτέλεση του Query 3 έδειξε σημαντική διαφορά απόδοσης μεταξύ των διαφορετικών μορφών αποθήκευσης δεδομένων.

Η χρήση CSV αρχείων παρουσίασε μεγαλύτερο χρόνο εκτέλεσης, καθώς το Spark απαιτεί πλήρη ανάγνωση και parsing όλων των δεδομένων. Αντίθετα, η χρήση Parquet επέτρεψε στο Spark να διαβάσει μόνο τις απαραίτητες στήλες, μειώνοντας σημαντικά το I/O overhead.

Η εφαρμογή της τεχνικής **partition pruning** οδήγησε σε ακόμα μεγαλύτερη βελτίωση της απόδοσης. Με την τεχνική αυτή, το Spark διαβάζει μόνο τα partitions που ικανοποιούν τα κριτήρια του query, αποφεύγοντας την επεξεργασία περιττών δεδομένων.

Επιπλέον, από την ανάλυση του Physical Plan προκύπτει ότι ο Spark επέλεξε τη στρατηγική **BroadcastHashJoin**, η οποία είναι ιδιαίτερα αποδοτική όταν το ένα από τα datasets είναι μικρού μεγέθους. Σε αυτή την περίπτωση, ο πίνακας περιοχών (lookup table) μεταδόθηκε σε όλους τους executors, αποφεύγοντας τη διαδικασία shuffle.

Συμπέρασμα

Η χρήση της μορφής αποθήκευσης Parquet σε συνδυασμό με τεχνικές βελτιστοποίησης, όπως το partition pruning και το BroadcastHashJoin, οδηγεί σε σημαντική μείωση του χρόνου εκτέλεσης του Query 3.

Η μετάβαση από CSV σε Parquet και η εφαρμογή partition pruning μπορούν να μειώσουν δραστικά τον χρόνο εκτέλεσης και να βελτιώσουν την αποδοτικότητα του συστήματος.

6. Q4 – Join με Projection και Aggregation

Το Query 4 έχει ως στόχο τον υπολογισμό στατιστικών με βάση τις γεωγραφικές περιοχές επιβίβασης (PickupBorough). Για την εκτέλεση του query πραγματοποιήθηκε join μεταξύ του dataset διαδρομών και του πίνακα taxi_zone_lookup, ώστε να ανακτηθούν πληροφορίες για την περιοχή κάθε διαδρομής.

Στη συνέχεια υπολογίστηκαν:

- το συνολικό πλήθος διαδρομών (Trips)
- το ποσοστό πληρωμών με κάρτα (card_share)
- ο μέσος λόγος φιλοδωρήματος προς κόμιστρο (avg_tip_rate_card)

Το query υλοποιήθηκε με χρήση του Spark SQL API και εκτελέστηκε τόσο σε δεδομένα μορφής CSV όσο και σε δεδομένα μορφής Parquet, με στόχο τη σύγκριση της απόδοσης των δύο μορφών αποθήκευσης.

Οι χρόνοι εκτέλεσης καταγράφηκαν μέσω του Spark History Server.

Χρόνος Εκτέλεσης

Υλοποίηση	**Χρόνος**
Q4_SQL_CSV	14 min
Q4_SQL_Parquet	32 s
Q4_SQL_Parquet (run 2)	30 s

Ανάλυση

Η εκτέλεση του Query 4 έδειξε σημαντική διαφορά απόδοσης μεταξύ των δύο μορφών αποθήκευσης δεδομένων.

Η χρήση CSV αρχείων οδηγεί σε μεγαλύτερο χρόνο εκτέλεσης, καθώς το Spark απαιτεί πλήρη ανάγνωση και parsing όλων των δεδομένων. Αντίθετα, το Parquet format επιτρέπει την ανάγνωση μόνο των απαραίτητων στηλών, μειώνοντας σημαντικά το I/O overhead.

Επιπλέον, από την ανάλυση του Physical Plan προκύπτει ότι ο Spark Catalyst Optimizer εφαρμόζει βελτιστοποιήσεις εκτέλεσης, όπως:

- projection pushdown
- optimized aggregation
- efficient join execution

Η χρήση του Parquet σε συνδυασμό με το Spark SQL API οδήγησε σε σημαντική μείωση του χρόνου εκτέλεσης.

Συμπέρασμα

Η χρήση του Spark SQL σε συνδυασμό με αρχεία Parquet αποτελεί την πιο αποδοτική προσέγγιση για την εκτέλεση του Query 4. Η επιλογή κατάλληλου storage format επηρεάζει καθοριστικά την απόδοση των queries και μπορεί να οδηγήσει σε δραστική μείωση του χρόνου εκτέλεσης.

7. Q5 – Imbalance ανά Zone και Ανάλυση Skew

Το Query 5 έχει ως στόχο τον υπολογισμό του δείκτη ανισορροπίας (imbalance) για κάθε γεωγραφική ζώνη (Zone). Ο δείκτης imbalance ορίζεται ως ο λόγος μεταξύ του μεγαλύτερου πλήθους διαδρομών (pickups ή dropoffs) και του μικρότερου πλήθους διαδρομών για την ίδια ζώνη.

Για την υλοποίηση του query χρησιμοποιήθηκε το DataFrame API του Spark και πραγματοποιήθηκε join μεταξύ των δεδομένων διαδρομών και του πίνακα taxi_zone_lookup, ώστε να ανακτηθούν πληροφορίες για τη ζώνη και τον δήμο (Borough).

Το query εκτελέστηκε σε δύο παραλλαγές:

1. Με default ρυθμίσεις Spark
2. Με χρήση hint / αλλαγή configuration ώστε να αλλάξει η στρατηγική join

Οι χρόνοι εκτέλεσης καταγράφηκαν μέσω του Spark History Server.

Χρόνος Εκτέλεσης

Υλοποίηση	Join Strategy	Χρόνος
Q5_DF_Default	BroadcastHashJoin	34 s
Q5_DF_Hint	SortMergeJoin	41 s

Ανάλυση Join Strategy

Στην πρώτη εκτέλεση, με default ρυθμίσεις, ο Spark Catalyst Optimizer επέλεξε:

BroadcastHashJoin

Η επιλογή αυτή είναι αναμενόμενη, καθώς ο πίνακας taxi_zone_lookup είναι μικρού μεγέθους και μπορεί να μεταδοθεί (broadcast) σε όλους τους executors χωρίς σημαντικό κόστος.

Στη δεύτερη εκτέλεση, χρησιμοποιήθηκε hint ή configuration αλλαγή, όπως:

spark.sql.autoBroadcastJoinThreshold = -1

ώστε να απενεργοποιηθεί το broadcast join.

Σε αυτή την περίπτωση, ο Spark επέλεξε:

SortMergeJoin

Η στρατηγική αυτή απαιτεί shuffle των δεδομένων και ταξινόμηση πριν από την εκτέλεση του join, γεγονός που αυξάνει τον χρόνο εκτέλεσης.

Ανάλυση Skew

Κατά την εκτέλεση του Query 5 παρατηρήθηκε skew σε συγκεκριμένες ζώνες με πολύ μεγάλο αριθμό διαδρομών, όπως:

- Manhattan
- JFK Airport
- LaGuardia Airport

Οι ζώνες αυτές εμφανίζουν σημαντικά μεγαλύτερο αριθμό pickups και dropoffs σε σχέση με άλλες περιοχές, γεγονός που οδηγεί σε άνιση κατανομή δεδομένων μεταξύ των partitions.

Το skew μπορεί να επηρεάσει αρνητικά την απόδοση του query, καθώς ορισμένα executors επεξεργάζονται πολύ μεγαλύτερο όγκο δεδομένων από άλλους.

Η χρήση broadcast join μειώνει την επίδραση του skew, καθώς αποφεύγεται το shuffle μεγάλου όγκου δεδομένων.

Συμπέρασμα

Η χρήση BroadcastHashJoin αποδείχθηκε πιο αποδοτική για το συγκεκριμένο query, καθώς ο πίνακας taxi_zone_lookup είναι μικρού μεγέθους και μπορεί να μεταδοθεί εύκολα στους executors.

Η απενεργοποίηση του broadcast join οδήγησε σε χρήση SortMergeJoin και αύξηση του χρόνου εκτέλεσης λόγω της ανάγκης για shuffle και sorting των δεδομένων.

Επιπλέον, παρατηρήθηκε skew σε συγκεκριμένες γεωγραφικές ζώνες με υψηλή κίνηση, γεγονός που μπορεί να επηρεάσει την απόδοση των distributed computations.

8. Q6 – Horizontal και Vertical Scaling

Το Query 6 έχει ως στόχο τον υπολογισμό βασικών οικονομικών δεικτών ανά VendorID και pickup service_zone. Συγκεκριμένα υπολογίστηκαν:

- Trips: πλήθος διαδρομών
- TotalRevenue: συνολικό ποσό διαδρομών
- CongestionAirport: σύνολο χρεώσεων congestion surcharge και airport fee
- Share: ποσοστό των χρεώσεων αυτών επί του συνολικού ποσού
- AvgRevenuePerTrip: μέσο έσοδο ανά διαδρομή

Για την εκτέλεση του query πραγματοποιήθηκε join μεταξύ του dataset διαδρομών και του πίνακα taxi_zone_lookup, ώστε να ανακτηθεί το service_zone για κάθε διαδρομή.

Στο συγκεκριμένο query πραγματοποιήθηκαν πειράματα scaling με διαφορετικές κατανομές πόρων εκτέλεσης, διατηρώντας σταθερό το συνολικό σύνολο πόρων:

- **8 cores**
- **16 GB memory**

Οι εκτελέσεις πραγματοποιήθηκαν με διαφορετικές διαμορφώσεις executors, ώστε να μελετηθεί η επίδραση του horizontal και vertical scaling στην απόδοση.

Configurations Scaling

Configuration	Executors	Cores per Executor	Memory per Executor	Type
Config 1	2	4	8 GB	Vertical scaling
Config 2	4	2	4 GB	Balanced scaling
Config 3	8	1	2 GB	Horizontal scaling

Χρόνος Εκτέλεσης

Configuration	Χρόνος
2 executors × 4 cores	29 s
4 executors × 2 cores	27 s
8 executors × 1 core	35 s

Ανάλυση

Τα αποτελέσματα δείχνουν ότι η κατανομή των πόρων επηρεάζει σημαντικά την απόδοση του query.

Η διαμόρφωση με:

4 executors × 2 cores

παρουσίασε την καλύτερη απόδοση, καθώς εξισορροπεί:

- τον βαθμό παραλληλισμού
- τη χρήση μνήμης
- το overhead επικοινωνίας μεταξύ executors

Η διαμόρφωση με:

8 executors × 1 core

παρουσίασε μεγαλύτερο χρόνο εκτέλεσης, λόγω αυξημένου communication overhead και μεγαλύτερου αριθμού tasks που απαιτούν συντονισμό.

Αντίθετα, η διαμόρφωση με:

2 executors × 4 cores

παρουσίασε καλή απόδοση, αλλά μικρότερο βαθμό παραλληλισμού σε σχέση με τη balanced διαμόρφωση.

Partitioning και Caching

Το dataset ήταν αποθηκευμένο σε μορφή Parquet με partitioning ανά pickup_date, γεγονός που επέτρεψε στο Spark να εφαρμόσει:

partition pruning

και να διαβάσει μόνο τα απαραίτητα δεδομένα.

Επιπλέον, η χρήση caching σε ενδιάμεσα DataFrames βελτίωσε την απόδοση σε επαναλαμβανόμενες εκτελέσεις του query.

Join Strategy

Από την ανάλυση του Physical Plan μέσω:

result.explain(True)

προκύπτει ότι ο Spark Catalyst Optimizer επέλεξε:

BroadcastHashJoin

Η επιλογή αυτή είναι λογική, καθώς ο πίνακας taxi_zone_lookup είναι μικρού μεγέθους και μπορεί να μεταδοθεί εύκολα σε όλους τους executors.

Η χρήση broadcast join μειώνει σημαντικά το shuffle των δεδομένων και βελτιώνει την απόδοση του query.

Συμπέρασμα

Η κατανομή των πόρων εκτέλεσης επηρεάζει σημαντικά την απόδοση των distributed queries.

Η balanced διαμόρφωση:

4 executors × 2 cores

παρουσίασε την καλύτερη απόδοση, καθώς συνδυάζει επαρκή παραλληλισμό με χαμηλό communication overhead.

Η χρήση Parquet format, partitioning και broadcast join συνέβαλε στη βελτιστοποίηση της εκτέλεσης του Query 6 και στη μείωση του χρόνου επεξεργασίας.

9. Μελέτη Catalyst Optimizer και Στρατηγικών Join

Ο Spark SQL χρησιμοποιεί τον Catalyst Optimizer για τη βελτιστοποίηση των queries πριν την εκτέλεσή τους. Ο optimizer αναλύει το logical plan ενός query και επιλέγει την πιο αποδοτική στρατηγική εκτέλεσης με βάση:

- το μέγεθος των datasets
- τη δομή των δεδομένων
- τις διαθέσιμες ρυθμίσεις του συστήματος
- τα στατιστικά στοιχεία των δεδομένων

Μία από τις σημαντικότερες βελτιστοποιήσεις είναι η επιλογή της στρατηγικής join, όπως:

- BroadcastHashJoin
- SortMergeJoin
- ShuffleHashJoin

Για τη μελέτη της επίδρασης του Catalyst Optimizer, εξετάστηκαν τα joins που εμφανίζονται στα Query 3, 4, 5 και 6, χρησιμοποιώντας την εντολή:

result.explain("formatted")

και τα logs του Spark History Server.

Query 3 – Join μεταξύ Trips και taxi_zone_lookup

Στο Query 3 πραγματοποιήθηκαν δύο joins:

- Pickup Location → Borough
- Dropoff Location → Borough

Ο Spark Catalyst Optimizer επέλεξε:

BroadcastHashJoin

Η επιλογή αυτή δικαιολογείται από το γεγονός ότι ο πίνακας:

taxi_zone_lookup

είναι μικρού μεγέθους και μπορεί να μεταδοθεί (broadcast) σε όλους τους executors.

Με τη στρατηγική αυτή αποφεύγεται η διαδικασία shuffle των δεδομένων, γεγονός που μειώνει σημαντικά τον χρόνο εκτέλεσης.

Query 4 – Join για υπολογισμό στατιστικών ανά Borough

Στο Query 4 πραγματοποιήθηκε join μεταξύ:

Trips dataset
και
taxi_zone_lookup

Ο Catalyst Optimizer επέλεξε:

BroadcastHashJoin

Η επιλογή αυτή είναι σύμφωνη με τη θεωρία, καθώς:

- το lookup table είναι μικρό
- το κύριο dataset είναι μεγάλο
- το broadcast μειώνει το κόστος επικοινωνίας

Το Physical Plan επιβεβαιώνει ότι το μικρό dataset μεταδόθηκε σε όλους τους executors.

Query 5 – Default vs Hint Join Strategy

Στο Query 5 πραγματοποιήθηκαν δύο εκτελέσεις:

Default execution

Ο Spark επέλεξε:

BroadcastHashJoin

καθώς το dataset taxi_zone_lookup είναι μικρού μεγέθους.

Execution με αλλαγή configuration

Χρησιμοποιήθηκε η ρύθμιση:

spark.sql.autoBroadcastJoinThreshold = -1

ώστε να απενεργοποιηθεί το broadcast join.

Σε αυτή την περίπτωση ο Spark επέλεξε:

SortMergeJoin

Η στρατηγική αυτή απαιτεί:

- shuffle των δεδομένων
- sorting πριν το join

γεγονός που αύξησε τον χρόνο εκτέλεσης του query.

Query 6 – Join για υπολογισμό revenue metrics

Στο Query 6 πραγματοποιήθηκε join μεταξύ:

Trips dataset
και
taxi_zone_lookup

Ο Catalyst Optimizer επέλεξε:

BroadcastHashJoin

Η επιλογή αυτή είναι λογική, καθώς:

- το lookup table είναι μικρό
- το κύριο dataset είναι μεγάλο
- η χρήση broadcast μειώνει το shuffle

Η στρατηγική αυτή συνέβαλε στη βελτίωση της απόδοσης του query κατά την εκτέλεση των scaling experiments.

Σύγκριση Στρατηγικών Join

Query	Join Strategy	Αιτιολόγηση
Q3	BroadcastHashJoin	μικρό lookup table
Q4	BroadcastHashJoin	μικρό lookup table
Q5 Default	BroadcastHashJoin	default optimizer choice
Q5 Hint	SortMergeJoin	broadcast disabled
Q6	BroadcastHashJoin	μικρό lookup table

Συμπέρασμα

Ο Catalyst Optimizer επιλέγει δυναμικά τη στρατηγική join με βάση το μέγεθος των δεδομένων και τις διαθέσιμες ρυθμίσεις.

Η χρήση BroadcastHashJoin αποδείχθηκε η πιο αποδοτική στρατηγική για τα περισσότερα queries της εργασίας, καθώς ο πίνακας taxi_zone_lookup είναι μικρού μεγέθους και μπορεί να μεταδοθεί εύκολα σε όλους τους executors.

Η απενεργοποίηση του broadcast join οδήγησε στη χρήση SortMergeJoin, η οποία απαιτεί shuffle και sorting των δεδομένων και συνεπώς αυξάνει τον χρόνο εκτέλεσης.

10. Συνολικά Συμπεράσματα

Στην παρούσα εργασία πραγματοποιήθηκε υλοποίηση και αξιολόγηση διαφορετικών τεχνικών επεξεργασίας μεγάλων δεδομένων με χρήση του Apache Spark και του Hadoop Distributed File System (HDFS). Τα ερωτήματα του Πίνακα 1 υλοποιήθηκαν με χρήση των APIs:

- RDD API (MapReduce λογική)
- DataFrame API
- Spark SQL

και εκτελέστηκαν τόσο σε δεδομένα μορφής CSV όσο και σε δεδομένα μορφής Parquet.

Η εργασία περιλάμβανε επίσης τη μελέτη της επίδρασης διαφορετικών στρατηγικών join και της κλιμάκωσης πόρων εκτέλεσης, με στόχο τη βελτιστοποίηση της απόδοσης των queries.

Κύρια Ευρήματα

1. CSV vs Parquet

Η μετατροπή των δεδομένων από CSV σε Parquet οδήγησε σε σημαντική βελτίωση της απόδοσης των queries.

Η μορφή Parquet:

- μειώνει το μέγεθος των δεδομένων
- επιτρέπει columnar storage
- υποστηρίζει predicate pushdown
- μειώνει το I/O κόστος

Τα queries που εκτελέστηκαν σε Parquet παρουσίασαν σημαντικά μικρότερο χρόνο εκτέλεσης σε σχέση με τα αντίστοιχα queries σε CSV.

2. Partitioning και Partition Pruning

Η αποθήκευση των δεδομένων σε μορφή Parquet με partitioning ανά:

pickup_date

επέτρεψε στο Spark να εφαρμόσει:

partition pruning

και να διαβάσει μόνο τα απαραίτητα δεδομένα.

Η τεχνική αυτή οδήγησε σε σημαντική μείωση του χρόνου εκτέλεσης, ιδιαίτερα σε queries που περιλάμβαναν φίλτρα ημερομηνίας.

3. Catalyst Optimizer και Join Strategies

Ο Spark Catalyst Optimizer επέλεξε δυναμικά τη στρατηγική join με βάση το μέγεθος των δεδομένων και τις ρυθμίσεις του συστήματος.

Στα περισσότερα queries επιλέχθηκε:

BroadcastHashJoin

καθώς ο πίνακας:

taxi_zone_lookup

είναι μικρού μεγέθους.

Η στρατηγική αυτή αποδείχθηκε ιδιαίτερα αποδοτική, καθώς μειώνει την ανάγκη για shuffle των δεδομένων.

Όταν απενεργοποιήθηκε το broadcast join, ο Spark επέλεξε:

SortMergeJoin

γεγονός που αύξησε τον χρόνο εκτέλεσης λόγω της ανάγκης για sorting και data shuffling.

4. Horizontal και Vertical Scaling

Η εκτέλεση του Query 6 με διαφορετικές διαμορφώσεις executors έδειξε ότι η κατανομή των πόρων επηρεάζει σημαντικά την απόδοση των distributed computations.

Η balanced διαμόρφωση:

4 executors × 2 cores

παρουσίασε την καλύτερη απόδοση, καθώς:

- εξασφαλίζει επαρκή παραλληλισμό
- μειώνει το communication overhead
- αξιοποιεί καλύτερα τη διαθέσιμη μνήμη

Η χρήση υπερβολικά πολλών executors (horizontal scaling) μπορεί να αυξήσει το overhead, ενώ η χρήση λίγων executors (vertical scaling) περιορίζει τον βαθμό παραλληλισμού.

5. Skew και Κατανομή Δεδομένων

Σε ορισμένα queries παρατηρήθηκε skew σε συγκεκριμένες γεωγραφικές ζώνες με μεγάλο αριθμό διαδρομών.

Η άνιση κατανομή δεδομένων μεταξύ partitions μπορεί να οδηγήσει σε:

- καθυστερήσεις στην εκτέλεση
- άνιση χρήση πόρων
- αυξημένο χρόνο ολοκλήρωσης

Η χρήση broadcast join και partitioning βοήθησε στη μείωση της επίδρασης του skew.

Τελικό Συμπέρασμα

Η εργασία ανέδειξε τη σημασία της σωστής επιλογής μορφής αποθήκευσης δεδομένων, στρατηγικών εκτέλεσης και κατανομής πόρων σε περιβάλλοντα επεξεργασίας μεγάλων δεδομένων.

Η χρήση:

- Parquet format
- partitioning
- broadcast joins
- κατάλληλου scaling

οδήγησε σε σημαντική βελτίωση της απόδοσης των queries και της αποδοτικότητας του συστήματος.

Τα αποτελέσματα δείχνουν ότι η βελτιστοποίηση της επεξεργασίας δεδομένων δεν εξαρτάται μόνο από τον κώδικα, αλλά και από:

- τη δομή των δεδομένων
- τη μορφή αποθήκευσης
- τις ρυθμίσεις εκτέλεσης
- τον σχεδιασμό της αρχιτεκτονικής του συστήματος